Liquid Coherence:

A Protocol for Codebook Alignment at System Boundaries - DRAFT

Ron Itelman
ron@ronitelman.com

March 2026

Abstract

Every framework that minimizes uncertainty—Shannon's channel coding, active inference, Bayesian decision theory—presupposes that sender and receiver share a codebook. We prove that no quantity computable from Shannon's model detects codebook misalignment, and show that this gap extends to every downstream framework. The receiver proceeds with zero uncertainty about an interpretation that may be wrong—a condition we call *ignorance of incoherence*.

We provide the instrument that fills the gap: a coherence protocol that decomposes the codebook into three comparison facets—Data, Meaning, and Structure—and a resolution layer, Context, that functions asymmetrically as the accumulating record of what was missing, surfaced, and resolved across the other three. Context is not a fourth dimension alongside the others. It is the protocol's working memory: the structured trace of every incompleteness discovered and every question answered in the course of achieving coherence. The protocol is grounded in an irreducible five-column canonical claim form onto which any system can project its codebook at the moment of contact, requiring no prior agreement on terms. We define a dependency ordering (Context → Meaning → Structure → Data), show that each comparison facet contributes up to 1 bit of measurable boundary uncertainty per field, and that Context may require an arbitrary number of resolution steps to ground those comparisons—uncertainty that compounds to an unbounded number of possible misalignment states across K boundaries, all invisible to standard metrics.

The protocol produces three actions—Halt, Ask, and Act—where Ask is a measurement in the formal sense: it acquires information that no computation on the received data can produce. Every Ask produces Context—a resolution record that may reveal further incompleteness, giving the protocol its recursive character. The cost is $O(1)$ per facet comparison per boundary crossing. The substrate reduces to binary and admits tensor decomposition, opening a path toward self-referential inference on the same surface that performs measurement.

1 Introduction

1.1 The Problem

Every framework that minimizes uncertainty—Shannon's channel coding Shannon [1948], active inference Friston [2010], Bayesian decision theory Savage [1954], Bellman [1957]—presupposes that sender and receiver share a codebook: a mapping between symbols and their referents. When this assumption is not verified, the receiver exists in a state we call *semantic superposition*: multiple internally consistent interpretations of the received data, with no information in the received signal to distinguish them. The receiver does not know it is in superposition. It silently collapses the ambiguity using its own defaults—its own timezone, its own definitions, its own reference frame—without verifying that the sender shares them, and proceeds with zero uncertainty about an interpretation that may be wrong.

The costs are not theoretical. Information theory underlies computer science, AI, and decision systems at every scale. When systems are incoherent without knowing it, the uncertainty does not disappear—it transfers into organizational and economic risk that no current instrument detects or prices. No standard information-theoretic or inferential framework provides a domain-agnostic instrument for detecting this condition at unmodeled boundaries. Shannon certifies the transmission as perfect Shannon [1948]; active inference reaches minimum free energy Friston [2010]; prediction error is zero. The answer is wrong. The problem sharpens as AI agents acquire autonomy and cross organizational boundaries outside any single system's authority—closed-system tests may pass while open-system failures accumulate undetected.

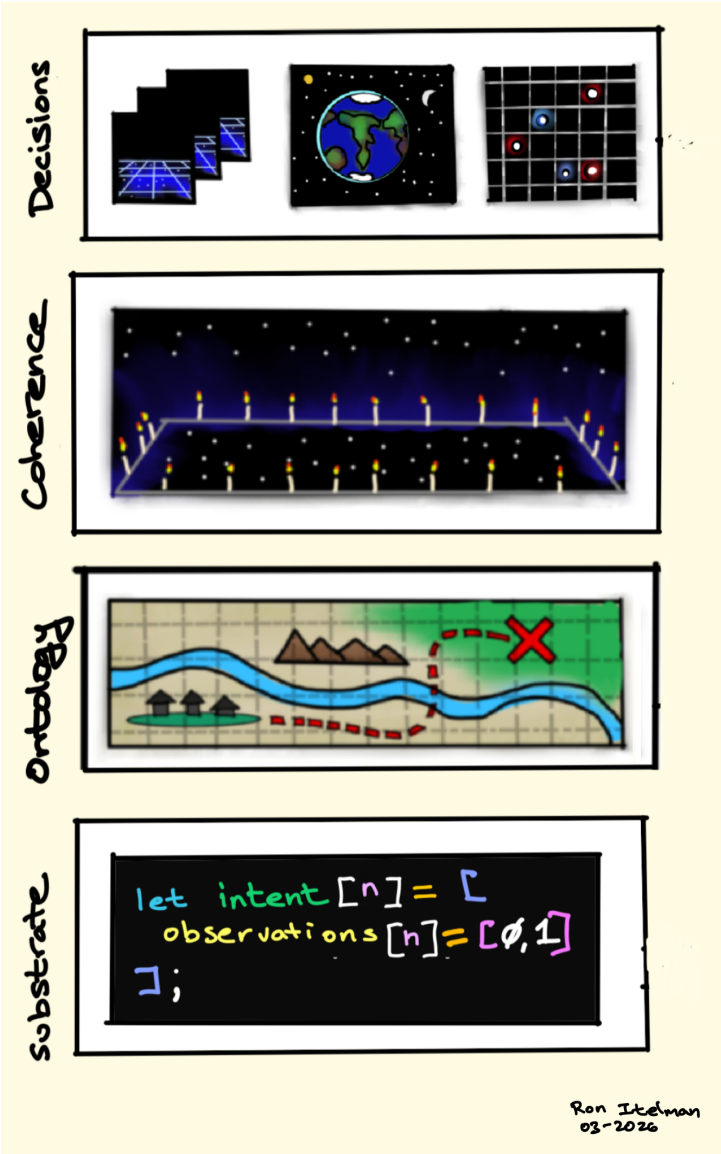

Figure 1: The four layers of the coherence architecture. **Decisions**: the systems that act on information. **Coherence**: the boundary measurement layer. **Ontology**: the domain knowledge that existing tools require but boundaries often lack. **Substrate**: the irreducible canonical claim form—observations reduced to $\{0, 1\}$, the binary foundation on which measurement, comparison, and inference converge.

1.2 Key Concepts

Shannon moved "information" from a general intuition to a formal measure Shannon [1948]. Context requires the same treatment. We introduce the following terms as used throughout this paper.

A *codebook* is the complete mapping a system maintains between its symbols and their referents—not merely a lookup table, but the full set of assumptions under which the system interprets data.

A *boundary* exists where two systems, each internally consistent under its own design authority, interact without authority over one another's ontology. Their codebooks may diverge in meaning, in structure, or in both—and without a standard measure, no party to the interaction can determine whether they diverge or agree.

A *facet* is one of four decomposable dimensions of a codebook: Data, Meaning, Structure, and Context Itelman and Viotti [2024]. These are not independent axes—they obey a dependency ordering formalized in later sections.

Context is the resolution layer for any incompleteness in any other facet. When Meaning is ambiguous, the resolution is Context. When Structure is underspecified, the resolution is Context. When neither Meaning nor Structure can be evaluated because reference frames have not been established, the prerequisite information is Context. Context is not a fourth kind of information alongside the other three—it is the structured record of what was missing, what was asked, what was resolved, and what remains open across Data, Meaning, and Structure. It is asymmetric with respect to the other three facets: Data, Meaning, and Structure describe what is being communicated; Context accumulates the resolutions required to interpret them. A match on Meaning or Structure without surfaced Context is unverifiable—not because Context is a separate dimension that also needs matching, but because the comparisons on Meaning and Structure are not yet well-defined.

Coherence is the state achieved when all necessary Context has been surfaced such that comparisons on Meaning, Structure, and Data are well-defined and have been evaluated. Two systems are coherent when the Context layer contains sufficient resolutions that no remaining ambiguity in Meaning, Structure, or Data is invisible to either party. Coherence is not a pre-existing condition discovered by inspection—it is the product of a measurement process. Two systems with identical codebooks that have not verified their alignment are not coherent in this sense. The protocol for making invisible context mutual and surfacable in an organized sequence is what produces coherence—this is why we call it a coherence protocol.

Ignorance of incoherence is the state in which a system acts on ambiguous information with full confidence because no instrument it possesses can distinguish the ambiguity from agreement. Semantic superposition describes the state of the information. Ignorance of incoherence describes the state of the system.

When AI agents, systems, or humans silently collapse the ambiguity at a boundary rather than measuring it, entropy is introduced and no single system observes it. Context mediation systems can reconcile known conflict types given a shared domain ontology Goh et al. [1999], Euzenat and Shvaiko [2007], but they require that both codebooks have been surfaced and modeled in advance. For boundaries where codebooks have not been surfaced—the default condition of most cross-system communication—no instrument exists. The problem has been acknowledged since 1949 Weaver [1949]. It remains open at the boundary layer.

1.3 Contribution

This paper provides the instrument: a coherence protocol for system boundaries. We call it a coherence protocol because the protocol produces coherence—it does not assume it or test for it after the fact. The protocol is the mechanism by which two systems move from an unmeasured state (semantic superposition) through measurement (externalizing and comparing

codebooks across facets) to an achieved state (coherence). The core operation is surfacing Context: the missing, incomplete, misinterpreted, or uninterpretable information that must be resolved before the codebook can be shared. If the resolution succeeds—if both sides arrive at a mutual understanding of what the data means, how it is structured, and under what conditions it was produced—the boundary is coherent. The protocol organizes this resolution into a dependency-ordered sequence.

We decompose the codebook into three comparison facets—Meaning, Structure, and Data—and a resolution layer, Context, that functions asymmetrically as the accumulating record of what was surfaced to ground comparisons on the other three. We define a canonical projection surface—a normalized representation onto which any system can externalize its codebook for comparison. We prove a dependency ordering: data comparison is semantically void until meaning and structure are aligned, and meaning and structure comparisons are unverifiable until Context has accumulated sufficient resolutions to make them well-defined.

The protocol produces three actions—Halt, Ask, and Act—governed by a decision structure, not a classification:

- **Halt**: a decidable condition required to proceed is not satisfied. This includes but is not limited to: codebooks cannot be surfaced (unmeasurable), measured uncertainty cannot be reduced below threshold (irreducible), or a deterministic system rule is violated (impermissible, e.g. PII in a query that prohibits it). No further protocol action at this boundary can resolve it.
- **Ask**: uncertainty is measurable and reducible. The divergence has been identified and can be narrowed through further exchange between systems.
- **Act**: all decidable conditions are met and measured uncertainty is below threshold. The boundary is coherent. Proceed.

We demonstrate the full calculation on a minimal example—one symbol, one boundary, three comparison facets plus Context—showing that a single communication event between Singapore and London carries up to 3 bits of comparison-facet boundary uncertainty per field plus Context resolution costs, invisible to Shannon and fully measurable by the gauge.

Every decision made on data whose codebook was assumed rather than verified—every financial comparison, every clinical inference, every autonomous action—carries undetectable risk. Systems report success. Decisions proceed with confidence. The confidence may be an artifact of silent misalignment. We invite rigorous scrutiny of these claims precisely because, if they hold, the implications extend to every system that acts on information crossing a boundary it did not design.

2 Grounding Example

To ground what follows, consider a minimal real-world case from finance. A stock market trade routing venue in Singapore transmits a single field to a stock trader in London, routed through the firm's headquarters system in New York. The message arrives. We begin with what the receiver now holds—the data—and work through each facet to show where ambiguity hides. We then run the coherence protocol without any formal machinery. The mathematics follow in the next section.

2.1 Data

Shannon's channel delivers the following perfectly. Every bit is correct. The receiver now holds the string `03/01/2026` under the label `time`. Shannon's metrics confirm perfect transmission: zero conditional entropy, maximal mutual information, zero bit error rate.

time
03/01/2026

The receiver is now in semantic superposition. No error flag, no warning, no signal of any kind indicates that ambiguity exists. Every instrument the receiver has reads clean. This is the condition the coherence protocol is built to detect: not incoherence, but *ignorance of incoherence*—a state in which a system acts on ambiguous information with full confidence precisely because no instrument it possesses can distinguish the ambiguity from agreement.

The four facets—Data, Meaning, Structure, and Context—are that instrument. Without them, the boundary is unmeasurable: not because the incoherence is absent, but because nothing decomposes the signal into the dimensions along which incoherence hides. We now apply each facet to this single field.

2.2 Structure

The sender's system validates `time` against the following schema:

```
{
  "time": {
    "type": "string",
    "format": "date",
    "pattern": "^\\d{2}/\\d{2}/\\d{4}$",
    "nullable": false
  }
}
```

The string `03/01/2026` passes validation. But the schema constrains *format*, not *interpretation*. It confirms that the string matches a date-like pattern. It does not specify whether the pattern is MM/DD/YYYY or DD/MM/YYYY. The value is structurally valid and semantically ambiguous.

2.3 Meaning

The sender defines `time` as:

> *The timestamp at which the order was placed by the customer.*

The receiver's system defines `time` as:

> *The timestamp at which the order was received by the fulfillment system.*

Both definitions are internally consistent. Both use the label `time`. They refer to different events. A comparison of `time` values between these systems would correlate two different moments in the lifecycle of every order—silently, with no error raised.

2.4 Context

Context is the resolution layer for incompleteness in any other facet. It is not a fourth kind of information alongside Data, Meaning, and Structure. It is the accumulating record of what was missing, what was asked, what was resolved, and what remains open across all three.

Each of the preceding facets surfaced an incompleteness. Structure specified a pattern (NN/NN/NNNN) but did not specify which position is month and which is day—that is an

incompleteness in Structure. The resolution ("the format is MM/DD/YYYY, because the producing system uses U.S. conventions") is Context. Meaning provided a label ("time") but two systems define it differently—that is an incompleteness in Meaning. The resolution ("time refers to order placement, not receipt") is Context. And neither resolution is possible without knowing which system produced the value and under what conditions—that prerequisite information is also Context.

Context is not limited to geography or time. It is whatever information is required to resolve incompleteness in the other three facets at a given boundary—and it is specific to that boundary. Two different boundaries will require different context, because the systems, the divergences, and the resolution paths differ.

Context is the only facet with a recursive property. As the protocol runs—as questions are asked, answers received, and facets updated—the record of what has been resolved and what remains open is itself Context. Each round of the protocol produces new Context that may reveal further incompleteness in another facet. Data, Meaning, and Structure can be updated through the protocol, but only Context tracks the state of the protocol itself. The accumulating structure of these dependencies forms a *context graph*—the state space the protocol traverses on its way to coherence or halt. This graph is not predefined. It is constructed as the protocol runs.

In this example, three reference frames are in play. The sender is located in Singapore (UTC+8). The receiver is located in London (UTC+0). The system that produced the data is hosted in New York (UTC−5). The string `03/01/2026` was generated by the New York system at 11:00 PM EST on February 28th. In Singapore, it was already March 1st. In London, it was 4:00 AM on March 1st. The date value is correct relative to the system that wrote it—but the sender, receiver, and system each occupy a different reference frame.

Each of these facts—the timezone, the locale, the producing system's conventions—is a resolution of an incompleteness in Structure, Meaning, or Data. Context does not add a fourth piece of information alongside the other three. It is the layer that accumulates the resolutions without which the other three cannot be interpreted. This is why Context functions asymmetrically: Data, Meaning, and Structure describe what is being communicated. Context resolves what is missing from each of them.

2.5 The Ignorance of Incoherence

Each of the three comparison facets—Data, Structure, and Meaning—passed its own local checks, and no Context had been surfaced to test whether those checks were valid. The data was transmitted without error. The schema validated. The definitions were internally consistent. The systems operated normally. Yet the receiver holds a value that is structurally ambiguous (date format), semantically misaligned (different definitions of `time`), and contextually ungrounded (unknown reference frame). The information is in semantic superposition. The system is in ignorance of incoherence. It proceeds with zero uncertainty—an artifact of having silently collapsed the superposition using its own defaults. The receiver did not measure. It assumed.

2.6 Running the Protocol

Before running the protocol, we summarize the measurement gap. Table 1 shows two instruments applied to the same field at the same boundary. One certifies delivery. The other finds the interpretation unresolved.

The measurement structure applies to this field as follows. μ_C is a completion check: have all resolutions required by the Meaning and Structure claims been recorded in Context? This is a binary precondition, not a semantic comparison. Once $\mu_C = 1$, two comparisons become well-defined: μ_M asks whether the parties assign the same referent to the symbol; μ_S asks whether

Table 1: The measurement gap: one field, one boundary, two instruments.

Shannon Channel Metrics	**Coherence Protocol Metrics**
Bit error rate: 0	Context: **unsurfaced** — incompleteness in Meaning (which event?), Structure (which format?), and reference frame (which timezone?) unresolved. No comparison on any facet is valid until these resolutions are recorded.
Conditional entropy: 0	Meaning: **misaligned** — two definitions of `time`, referring to different events
Mutual information: maximal	Structure: **ambiguous** — MM/DD/YYYY vs DD/MM/YYYY undetermined
Diagnosis: perfect transmission.	*Diagnosis: Context layer empty ($\mu_C = 0$). Conditional on accumulating sufficient resolutions, up to 3 additional bits of measurable boundary uncertainty: 1 bit each on Meaning, Structure, and Data (μ_M, μ_S, μ_D). Total: 3 bits from comparison facets plus resolution cost of Context.*
Protocol action: **Act**	Protocol action: **Ask**

they apply the same format and constraints. μ_D is the residual check—do the data values agree once meaning and structure are verified to be shared? Each comparison facet contributes up to 1 bit. Context itself may require an arbitrary number of resolution steps (timezone, locale, producing system, fiscal calendar, etc.) to reach $\mu_C = 1$—the number of Context bits is boundary-dependent, not fixed at one. The total measurable boundary uncertainty per field is 3 bits from the comparison facets plus however many resolution steps Context required. Shannon's channel metrics report zero residual uncertainty about the received symbol string itself.

Now we run the coherence protocol at this boundary, respecting the dependency ordering: accumulate Context first (precondition for valid comparison), then compare Meaning and Structure (comparison facets), then interpret Data.

Context (resolution step 1: reference frame). Three reference frames, no shared one. The incompleteness spans all three comparison facets—none can be evaluated without knowing which system's conventions apply. The protocol action is **Ask**. The system of record responds: UTC−5, MM/DD/YYYY. This resolution is recorded as Context. It resolves the structural incompleteness (format is MM/DD/YYYY) and establishes the reference frame for Meaning and Data.

Meaning. Two definitions of `time`, referring to different events. Now that Context has established the reference frame, this comparison is well-defined. The protocol action is **Ask**. The sender confirms: order placement time. The resolution is recorded as Context. Meaning is aligned.

Structure. With context accumulated, the format is resolved: MM/DD/YYYY, so `03/01/2026` is March 1st. If the receiver requires ISO 8601, the protocol action is **Ask** and the resolution is again recorded as Context. Structure is aligned.

Data. Only now is the value interpretable. All necessary Context has been accumulated.

The comparison facets are aligned. The protocol action is **Act**.

Before the protocol, the receiver had zero uncertainty and a potentially wrong answer. After the protocol, the receiver has zero uncertainty and a verified answer. The difference is not in the data. The difference is that Context now contains a complete record of resolutions: which system produced the value, what format it used, what event the field refers to. The codebook is shared because every incompleteness was surfaced, asked about, and resolved. Semantic superposition has collapsed—not by assumption, but by measurement.

Shannon measures the channel and certifies delivery. The coherence protocol measures the boundary and finds the interpretation unresolved. Both are correct. They measure different things. No instrument in standard use measures what the coherence protocol measures.

Figure 2 illustrates this gap across an entire year of observations. Even in the Data facet alone, 40% of days carry structural ambiguity that Shannon cannot detect.

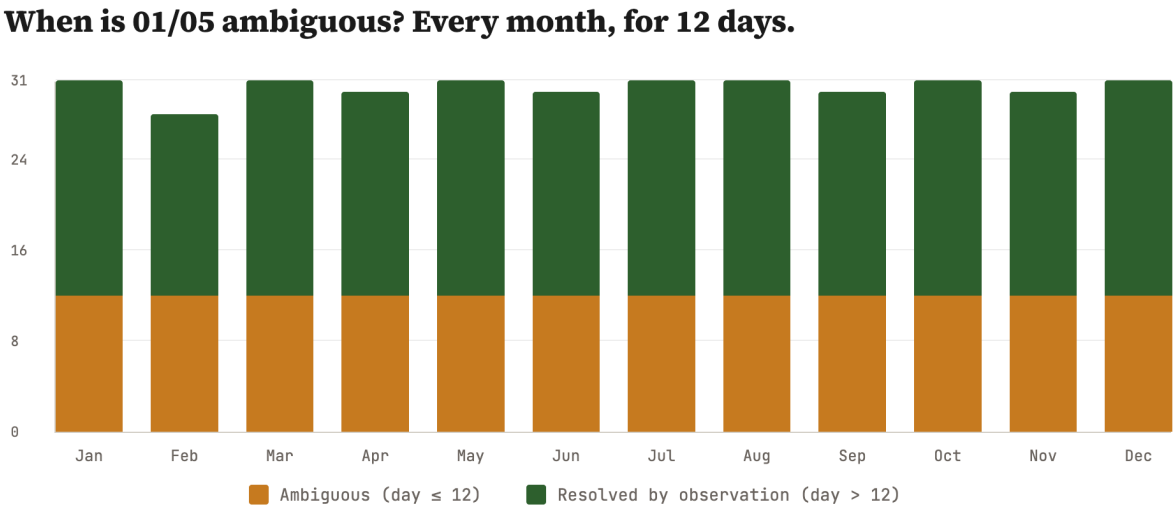

Figure 2: Structural ambiguity in the Data facet across a full year of observations. For any date formatted as MM/DD or DD/MM, days 1–12 are ambiguous to any receiver that does not share the sender's format convention. 12 of every month's days carry ambiguity that Shannon cannot detect—zero transmission error, unresolvable interpretation. Orange: ambiguous (day ≤ 12). Green: resolved by observation (day > 12).

This is the condition we address. It arises at every boundary between independently designed systems—every API call, every database join, every LLM retrieval, every multi-agent message—whenever a symbol crosses a boundary and the codebook that gives it meaning stays behind.

3 The Codebook Is Not on the Wire

The Grounding Example demonstrated the problem on one field. We now prove that no existing instrument can detect it.

A sender selects a symbol x from alphabet $\mathcal{X}$ and transmits it through a channel. A receiver observes y from alphabet $\mathcal{Y}$. Shannon's model is the tuple $(\mathcal{X}, \mathcal{Y}, p(x), p(y|x))$. Every Shannon quantity is a function of the joint distribution $p(x, y)$.

Let $\mathcal{R}$ be a referent space. The sender's codebook $C_A : \mathcal{X} \to \mathcal{R}$ maps symbols to referents. The receiver's codebook $C_B : \mathcal{X} \to \mathcal{R}$ maps the same symbols to potentially different referents. Neither C_A nor C_B appears in Shannon's model.

Observation 1 (The Codebook Is Not on the Wire). No quantity computable from $(\mathcal{X}, \mathcal{Y}, p(x), p(y|x))$ distinguishes aligned codebooks ($C_A = C_B$) from misaligned codebooks ($C_A \neq C_B$).

The argument is immediate: C_A and C_B do not appear in the tuple $(\mathcal{X}, \mathcal{Y}, p(x), p(y|x))$. The joint distribution $p(x, y)$ is identical whether $C_A = C_B$ or $C_A \neq C_B$. Every Shannon quantity is a function of $p(x, y)$. Therefore no Shannon quantity varies with codebook alignment.

This extends to every downstream framework. Active inference Friston [2010]: free energy is a function of the agent's own generative model; the data source's codebook does not appear. Bayesian decision theory Savage [1954]: expected utility is computed over states whose definitions may differ between source and model. Dynamic programming Bellman [1957]: the optimal policy is optimal for a formalization that may not correspond to the real-world problem. In each case, the framework's success metrics report that everything is working. No quantity any of them compute references the other side's codebook.

The consequence is precise: the entire suite of instruments the field uses to certify communication and inference as successful has no reading for codebook alignment. The Grounding Example showed what this looks like on one field. The next section provides the instrument that fills the gap.

4 The Four-Facet Gauge

A symbol crossing a boundary carries a value. A value alone is in semantic superposition. The codebook that collapses the superposition has internal structure. We decompose it into three comparison facets—Meaning, Structure, and Data—and a resolution layer, Context, that accumulates the information required to make comparisons on the other three well-defined.

Definition 2 (Codebook Decomposition)**.** A structured codebook for a symbol x is a tuple $C(x) = (d, m, s, c)$ where:

- $d \in \mathcal{R}_D$ is the **Data**: the raw value. This is what Shannon delivers.
- $m \in \mathcal{R}_M$ is the **Meaning**: the semantic definition—what the value refers to.
- $s \in \mathcal{R}_S$ is the **Structure**: the schema and constraints—how the value is encoded.
- $c \in \mathcal{R}_C$ is the **Context**: the accumulated resolutions of incompleteness in the other three.

Data, Meaning, and Structure are *comparison facets*: they describe what is being communicated and can be directly compared across systems. Context is the *resolution layer*. It is not a fourth dimension to be matched—it is the accumulating record of what was missing from Data, Meaning, or Structure and what was surfaced to fill the gap. Each party externalizes what it needed to resolve (timezone, format convention, definitional scope, producing system) so that comparisons on Meaning and Structure become well-defined. A match on Meaning or Structure without sufficient accumulated Context is unverifiable.

Definition 3 (Required Context Declarations)**.** Each Meaning and Structure claim in the canonical form may declare the context fields required for valid comparison. A required context declaration is a canonical claim row of the form:

`urn:context:required:`*`facet_type`*`:`*`field_id`*

where *facet_ type* $\in$ {`meaning`, `structure`} identifies the facet being conditioned on, and *field_ id* identifies the specific property (e.g., `urn:context:required:meaning:def` $\rightarrow$ `urn:context:timezone`). A system may declare multiple required context fields for a single Meaning or Structure claim as separate rows. The set of required context fields for a claim is determined locally by the system that produces it; no prior agreement on which fields are required is assumed.

Definition 4 (Facet Mismatch Indicators)**.** Given systems A and B with structured codebooks $C_A(x) = (d_A, m_A, s_A, c_A)$ and $C_B(x) = (d_B, m_B, s_B, c_B)$, the four facet indicators are:

$$\mu_C(x) = \mathbf{1}\big[\text{all } \texttt{urn:context:required:*} \text{ fields declared by } C_A(x) \text{ and } C_B(x) \text{ are present in both projections}\big] \tag{1}$$

$$\mu_M(x) = \mathbf{1}[m_A \neq m_B] \tag{2}$$

$$\mu_S(x) = \mathbf{1}[s_A \neq s_B] \tag{3}$$

$$\mu_D(x) = \mathbf{1}[d_A \neq d_B] \tag{4}$$

Context is not matched for equality—μ_C is a completion check on the resolution layer: have all resolutions required by the Meaning and Structure claims been recorded? This is binary by construction: either sufficient Context has been accumulated (1) or it has not (0). μ_M and μ_S are valid comparisons only when $\mu_C = 1$. μ_D is interpretable only when $\mu_C = 1$, $\mu_M = 0$, and $\mu_S = 0$. Each comparison indicator (μ_M, μ_S, μ_D) contributes up to 1 bit of boundary uncertainty. Context may require an arbitrary number of resolution steps to reach $\mu_C = 1$—the cost of grounding is boundary-dependent. The total measurable boundary uncertainty per field is 3 bits from the comparison facets plus the resolution cost of Context.

Definition 5 (Dependency Ordering)**.** Data comparison is semantically void unless Meaning and Structure are aligned. Meaning and Structure comparisons are unverifiable unless Context has been surfaced.

The reasoning is direct. If Meaning differs, the values measure different concepts; their difference has no shared referent. If Structure differs, the values are in different formats; their difference confounds discrepancy with transformation. And two systems may both define `time` as "order placement time" and both use MM/DD/YYYY—but if one operates in UTC+8 and the other in UTC−5, the same string refers to different calendar days. Meaning matched. Structure matched. The answer is wrong. Context determines whether matches are real.

The resolution ordering is:

$$\text{Check } \mu_C \rightarrow \text{Compare Meaning} \rightarrow \text{Compare Structure} \rightarrow \text{Interpret Data} \tag{5}$$

Each step in this ordering contributes one bit of boundary uncertainty:

Indicator	**Question**	**Valid when**
μ_C	Have all resolutions required by Meaning and Structure claims been recorded in Context?	Always—this is the completion check on the resolution layer
μ_M	Do the meaning definitions match?	$\mu_C = 1$
μ_S	Do the format/schema definitions match?	$\mu_C = 1$
μ_D	Do the data values agree under aligned codebooks?	$\mu_C = 1$, $\mu_M = 0$, $\mu_S = 0$

The comparison facets (μ_M, μ_S, μ_D) each contribute up to 1 bit of measurable boundary uncertainty. Context is not a fourth comparison axis—it is the resolution layer whose completion is the precondition for the other three comparisons to be well-defined. The number of resolution steps required to reach $\mu_C = 1$ is boundary-dependent: a single timezone declaration may suffice at one boundary; another may require timezone, locale, fiscal calendar, regulatory jurisdiction, and producing system identity. μ_D is not an independent comparison of raw values—it is the residual question of whether two values differ once meaning and structure are verified to be shared.

Remark 6 (String Equality and URI Resolution). The gauge compares Meaning and Structure claims by string equality—the simplest possible operation. This is the cheapest first pass in a natural resolution cascade.

When two systems use different URIs for the same referent, the gauge reports mismatch—a safe false positive that triggers Ask. The Ask may take several forms: dereference both URIs to their documentation pages and compare tagged definitions; submit both definitions to a semantic comparator (human, AI, or rule-based); or query a shared registry. Each form acquires information not present in the original string comparison. Each produces Context.

The dangerous case is the reverse: same string, different referent. Both systems write `urn:meaning:def = "execution time"`, but one means "when the trade was executed at the venue" and the other means "when our system executed the processing job." The gauge reports match. This is precisely the case that the Context resolution layer addresses: required context declarations (Definition 3) prevent the protocol from reaching Act on a Meaning or Structure match until the conditions under which both sides assigned those strings have been surfaced and verified.

String equality is Level 1: $O(1)$, catches exact matches and obvious mismatches. URI dereferencing is Level 2: follow the link, compare tagged definitions. Semantic evaluation is Level 3: human, AI, or domain-expert judgment. Each level is more expensive and acquires more information. The protocol does not prescribe which level to use—it prescribes the dependency ordering and records every resolution as Context. The existing semantic web stack (RDF, OWL, SHACL) provides the infrastructure for making URIs dereferenceable and definitions machine-readable. The coherence protocol provides the decision structure for when to dereference, in what order, and what to do with the result.

4.1 ASK as Measurement

The dependency ordering is not merely a sequencing convenience. It reflects a mathematical constraint: some uncertainty is provably non-reducible by inference alone.

No computation on the string `09/24/2026` can determine whether it means "time the order was placed" or "time the order was fulfilled." No algorithm can determine whether the format is MM/DD/YYYY or DD/MM/YYYY. No model can determine whether the value was produced in Singapore or New York. The answer is not in the data. It is in the sender's codebook, on the other side of the boundary. Shannon certified the transmission as complete. The data is all there. The ambiguity persists.

Each Ask in the coherence protocol is therefore a *measurement* in the formal sense: it acquires information that cannot be computed from the data the receiver already holds. This is the distinction between inference—reducing uncertainty from data you have—and measurement—acquiring data you cannot compute. The protocol's Ask is the only operation that can collapse this class of uncertainty. And every Ask produces Context: a resolution record that becomes part of the protocol's working memory. The recursive property follows directly—each resolution may reveal further incompleteness, requiring further Asks, until the Context layer is complete and the comparison facets can be evaluated.

We demonstrate the collapse sequence on a single field with 12 possible interpretations (3 meanings $\times$ 2 formats $\times$ 2 timezones). Each ASK eliminates interpretations that no amount of inference could have ruled out.

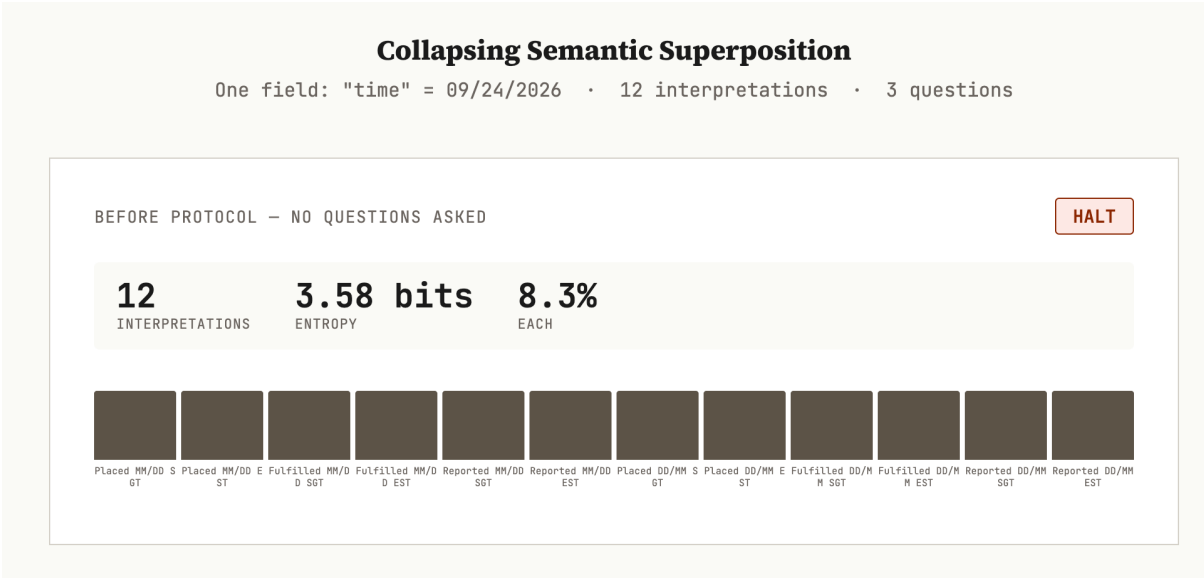

Figure 3: Before the protocol. One field (`time` = `09/24/2026`) admits 12 equally likely interpretations: 3 meanings (placed, fulfilled, reported) $\times$ 2 formats (MM/DD, DD/MM) $\times$ 2 timezones (SGT, EST). Entropy: $\log_2(12) = 3.58$ bits. Regime: Halt. No computation on the received string can reduce this.

Figure 4: ASK 1: Meaning. "What does `time` refer to?" The sender responds: time the order was fulfilled. 8 interpretations are eliminated. $12 \to 4$. Entropy: $3.58 \to 2.00$ bits. Information gained: 1.58 bits. This information was not in the data—it was in the sender's codebook.

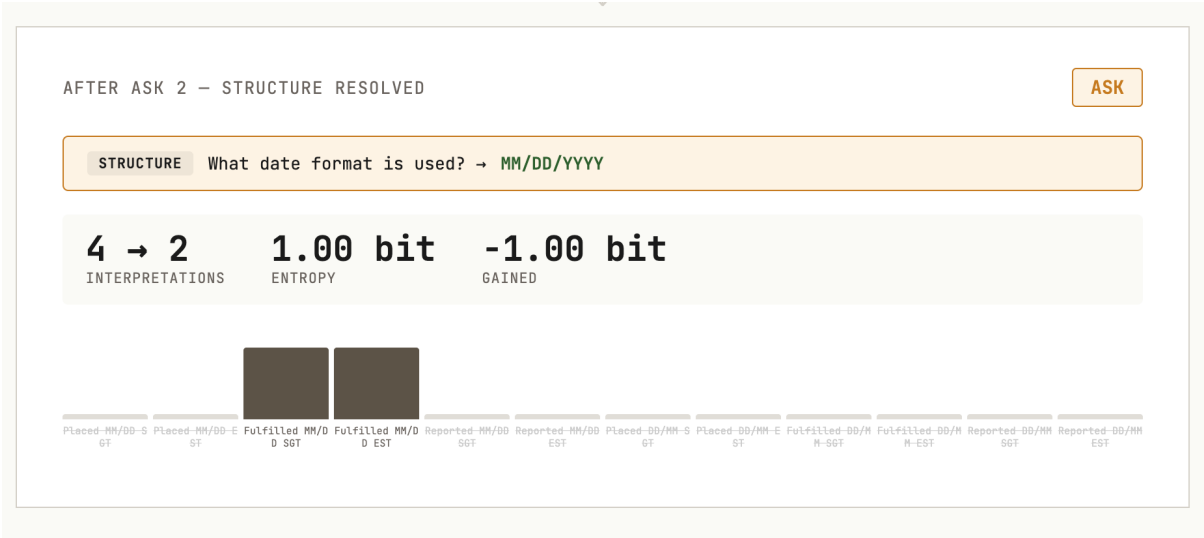

Figure 5: ASK 2: Structure. "What date format is used?" The sender responds: MM/DD/YYYY. 2 more interpretations eliminated. 4 → 2. Entropy: 2.00 → 1.00 bit. Information gained: 1.00 bit. Again, no algorithm applied to the string `09/24/2026` could have determined this—the day field happens to be > 12, but the format convention is a property of the sender's system, not the value.

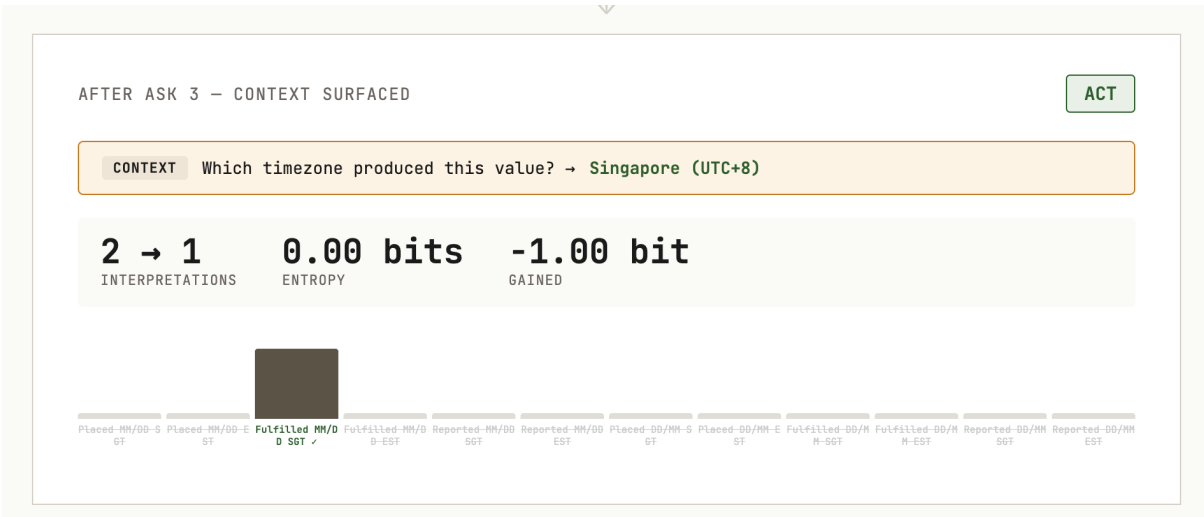

Figure 6: ASK 3: Context. "Which timezone produced this value?" The sender responds: Singapore (UTC+8). The final ambiguity collapses. 2 → 1. Entropy: 1.00 → 0.00 bits. Information gained: 1.00 bit. Regime: Act. The value `09/24/2026` now has a definite, shared referent: September 24th, 2026, fulfillment time, MM/DD/YYYY, Singapore local time.

Total: 3.58 bits of uncertainty, invisible to Shannon, collapsed to zero by three measurements. The protocol action transitions from Halt (before any ASK) through Ask (after each partial collapse) to Act (after full collapse). No inference on the received data—no matter how sophisticated—could have achieved any of these reductions. The information was on the other side of the boundary. The only way to get it was to ask.

WHY ASK IS A MEASUREMENT, NOT AN INFERENCE

No computation on the string `09/24/2026` can determine whether it means "placed" or "fulfilled," whether it is MM/DD or DD/MM, or whether it was produced in Singapore or New York. **The answer is not in the data.** It is in the sender's codebook, on the other side of the boundary. Each ASK is a sample from a distribution the receiver cannot access by computation. This is the formal distinction: inference reduces uncertainty from data you already have; measurement acquires data you cannot compute. The coherence protocol's ASK is a measurement in the formal sense — the only operation that can collapse this class of uncertainty.

Figure 7: Why ASK is measurement, not inference. The answer to each question is not in the received data—it is in the sender's codebook, on the other side of the boundary. Inference reduces uncertainty from data you have. Measurement acquires data you cannot compute. Each ASK is a sample from a distribution the receiver cannot access by computation. This is the class of uncertainty that the coherence protocol is built to collapse.

5 Propagation in Open Systems

The gauge measures one boundary. Data never crosses just one. We now trace a single trade record through five independently designed systems to show how boundary uncertainty compounds silently—and why the problem cannot be standardized away.

5.1 The Chain

A stock market trade routing venue in Singapore (UTC+8) executes a trade at 09:00 local time on Tuesday, March 10. The trade record includes a single field:

time	action
03/10/2026	BUY

This record will cross five boundaries before it comes to rest. At each one, the codebook changes. At none of them is the change detected.

Boundary 1: Trade Venue → Trader. The record arrives at a trader's terminal in London (UTC+0). The trader's system also uses MM/DD/YYYY—Structure matches. But the trader's system defines `time` as "time the order was received by our system," not "time the trade was executed at the venue"—Meaning mismatches. The timestamp on the trader's screen is 01:00 AM London time, still March 10th, but 8 hours behind the venue's clock. Context was never surfaced. The Meaning comparison is unverifiable. The trader proceeds. The timestamp is now silently redefined.

Boundary 2: Trader → Broker. The trader decides to buy more shares based on the trade they received. They submit a market order through their broker in New York (UTC−5). The broker timestamps the order at 20:00 EST on March 9th—the previous calendar day. The broker defines `time` as "time the order was submitted to the exchange."

The original trade happened on March 10th in Singapore. The buy decision happened on March 10th in London. The broker recorded it as March 9th in New York. Three systems, three calendar dates for events in the same causal chain. No system has flagged a discrepancy.

Boundary 3: Broker → Exchange. The exchange executes the order and returns a receipt in UTC, ISO 8601: `2026-03-10T01:00:00Z`. This is the first system in the chain that uses UTC and ISO 8601—Structure mismatches with the broker's MM/DD/YYYY in local time. The broker converts correctly: `03/09/2026 20:00 EST`. The conversion is lossless. But the broker's downstream reports now show a trade date of March 9th for an event that originated on March 10th in Singapore.

Boundary 4: Broker → SEC. The broker files the transaction with the SEC. The SEC expects timestamps in UTC, ISO 8601, with the field defined as "time of execution at the primary exchange." The broker's `time` field has been silently redefined twice: from "execution time" (Singapore) to "receipt time" (London) to "order submission time" (New York). The SEC receives a timestamp that is labeled correctly, formatted correctly, and refers to the wrong event. Every automated compliance check passes.

Boundary 5: SEC → Insurance Risk Model. An insurance company pulls SEC filings to build a risk model for the fund. The model correlates trade timestamps with market events to estimate volatility exposure. The model assumes all timestamps refer to execution time at the primary exchange—the SEC's definition. But this timestamp refers to order submission time at the broker, inherited from boundary 2.

The risk model contains a systematic timing error. Every trade that passed through this chain has its timestamp shifted by hours, potentially across a date boundary. The model's volatility estimates are computed on the wrong event sequence. The confidence intervals are precise. They are precisely wrong.

5.2 The Accumulation

Table 2: Propagation of boundary uncertainty through the chain.

Boundary	Systems	Meaning	Structure	Context
1	Venue → Trader	mismatch	match	unsurfaced
2	Trader → Broker	mismatch	match	unsurfaced
3	Broker → Exchange	match	mismatch	unsurfaced
4	Broker → SEC	mismatch	match	unsurfaced
5	SEC → Insurer	match	match	unsurfaced

Context was never surfaced at any boundary. Every Meaning and Structure comparison in the table is unverifiable—the matches may be coincidence, the mismatches may be worse than they appear. The resolution layer is empty at every hop: no system asked what the other system meant, what conventions it used, or under what conditions the data was produced. The total measurable boundary uncertainty, conditional on surfacing Context at each hop, is at least 3 bits per boundary from the comparison facets (Meaning, Structure, Data) plus however many resolution steps each boundary's Context layer would require. Shannon reports 0 at every hop. Every system in the chain reported success.

For K boundaries, considering only the three comparison facets, the state space of possible misalignment configurations across comparison facets alone is 2^{3K}. When Context resolution steps are included—each boundary potentially requiring multiple resolutions—the effective state space grows further. Even the comparison-facet bound is instructive:

Boundaries (K)	Comparison-facet states	Upper bound
1	8	3 bits
2	64	6 bits
5	$\sim 32{,}768$	15 bits
6	$\sim 262{,}144$	18 bits
10	$\sim 1.07 \times 10^9$	30 bits

With unresolved Context at each boundary, the effective state space is larger—each unsurfaced resolution multiplies the possible interpretations. A routine data pipeline with 6 boundaries has over 260,000 possible comparison-facet misalignment states before accounting for Context resolution costs. The coherence protocol collapses each boundary independently at $O(1)$ per facet comparison per boundary.

5.3 Why This Cannot Be Solved Once

An organization might respond: standardize everything. All systems use UTC, ISO 8601, and a single definition of `time`. This is the closed-system assumption. It does not hold.

Organizations are not closed systems. Internal terminology drifts constantly—Anthony Alcaraz, an AI strategist at AWS reported that internal vocabulary shifts approximately every two months, and "there needs to be a way for agents to resolve conflicts". Frameworks, methodologies, project names, and field definitions evolve as teams change. A field called `time` in a system designed in 2022 may carry a different operational definition than a field called `time` in a system designed in 2025, even within the same organization.

Foundation models compound the problem. A model trained in January operates with a codebook frozen at training time. By July, the organization's terminology has shifted. The model confidently uses the old definitions. An agent built on this model and given autonomy to execute trades, file regulatory reports, or build risk models is operating with a stale codebook—and has no instrument to detect the drift. In terms of the protocol: the agent's Context layer is empty. It has never surfaced what its terms mean relative to the current system. It proceeds in ignorance of incoherence.

The coherence protocol does not prevent drift. Drift is a property of open systems and cannot be eliminated. The protocol provides the instrument to detect drift at the moment a symbol crosses a boundary—to measure whether the codebook that produced the value is the same codebook that will interpret it. Context is the structured record of that measurement: every resolution recorded, every incompleteness surfaced. Without this instrument, every boundary in the chain has an empty Context layer: the appropriate action is Halt, but every system silently Acts.

6 The Canonical Claim Form

The gauge tells you *what to measure*—four facets, in dependency order. The canonical claim form tells you *how to represent any measurement*. It is the irreducible substrate onto which any system can project its codebook for comparison.

Definition 7 (Canonical Claim)**.** A *claim* is a tuple (e, a, t, k, v) where:

- e is a subject identifier (URI): what entity is this about?
- a is an asserting party (URI): who makes this claim?
- t is a timestamp: when?
- k is a property identifier: what property?
- v is an asserted value: what assertion?

Table 3: The canonical claim form—the irreducible substrate.

id	**source**	**timestamp**	**key**	**value**
What entity	Who asserts	When	What property	What assertion

Every assertion from every system, regardless of source format, reduces to a single row in this table. A JSON field is one row. A CSV cell is one row. A SHACL shape decomposes into one row per assertion. No column can be removed without losing information the protocol requires.

6.1 Facets in the Identifier

The four facets do not require separate tables. The facet is encoded in the URN namespace of the key:

Table 4: One flat log, self-organizing by facet.

id	**source**	**timestamp**	**key**	**value**
`urn:trade:001`	`urn:sys:sgx`	2026-03-10T09:00+08	`urn:data:value`	03/10/2026
`urn:trade:001`	`urn:sys:sgx`	2026-03-10T09:00+08	`urn:meaning:def`	execution time
`urn:trade:001`	`urn:sys:sgx`	2026-03-10T09:00+08	`urn:structure:fmt`	MM/DD/YYYY
`urn:trade:001`	`urn:sys:sgx`	2026-03-10T09:00+08	`urn:context:timezone`	UTC+8
`urn:trade:001`	`urn:sys:sgx`	2026-03-10T09:00+08	`urn:context:locale`	en-SG

One table. One log. The facet is the URN prefix (`urn:data:`, `urn:meaning:`, `urn:structure:`, `urn:context:`). The URN path supports hierarchies (`urn:context:timezone`, `urn:context:locale`, `urn:context:fiscal_calendar`), so any domain ontology can be expressed without changing the schema. The five columns are fixed. The URN namespace is open.

Any system can mint URIs locally—a UUID formatted as a URN (`urn:uuid:550e8400-e29b-...`) requires no central registry and no coordination. Two systems generating URIs independently will never collide. When an entity maps to a well-known concept, the global URI is recorded as a claim, not substituted for the local id.

6.2 Liquid Assembly

The canonical form is not a fixed schema. Neither side must adopt the other's representation. Each system projects its own codebook onto the five-column surface at the moment of contact. The surface assembles on demand from whatever each side brings—it is *liquid*, taking the shape of the boundary rather than imposing a shape on it.

This is the property that distinguishes the canonical form from a shared ontology. An ontology requires prior agreement on terms—it is solid, pre-agreed, and brittle when systems change. The canonical form requires only that both sides can project onto the same five-column surface. It is liquid: on-demand, boundary-specific, and tolerant of systems that evolve independently.

6.3 Comparison

Comparison across the canonical form is mechanical, following the dependency ordering:

1. **Accumulate Context.** Both systems project their `urn:context:*` claims—the resolutions required to ground Meaning and Structure comparisons. The protocol checks for completeness: have all resolutions declared as required by each side's Meaning and

Structure claims been recorded? Context is not compared for equality. It is checked for sufficiency.

2. **Compare Meaning.** URI equality on `urn:meaning:*` claims. Match or mismatch—up to 1 bit.
3. **Compare Structure.** URI equality on `urn:structure:*` claims. Match or mismatch—up to 1 bit.
4. **Interpret Data.** Only now. The `urn:data:*` value is read through aligned Meaning and Structure, grounded by surfaced Context.

6.4 The Decision Interface

The protocol does not classify boundaries into static regimes. It evaluates decidable conditions at each boundary and produces one of three actions:

- **Halt**: a decidable condition required to proceed is not satisfied. This includes but is not limited to: codebooks cannot be surfaced, measured uncertainty cannot be reduced below threshold, a deterministic system rule is violated, or any condition the boundary context requires that has not been met. The reasons for Halt are open-ended and context-dependent. The action is closed: do not proceed.
- **Ask**: uncertainty is measurable and reducible. The divergence has been identified and can be narrowed through further exchange. Each Ask produces new Context—a resolution record that fills an incompleteness in Meaning, Structure, or the prerequisites for evaluating them. This new Context may reveal further incompleteness, requiring further Asks. This is the recursive property: Context is the protocol's working memory, and each resolution may reshape what remains to be resolved.
- **Act**: all decidable conditions are met and measured uncertainty is below threshold. The boundary is coherent. Proceed.

The architecture separates measurement from judgment in three layers. **Layer 1 (Gauge):** URI equality on each facet—match or mismatch, no interpretation. **Layer 2 (Protocol):** the dependency ordering that determines which comparisons are valid and in what sequence. **Layer 3 (Decision Interface):** evaluation of all decidable conditions and selection of Halt, Ask, or Act. The gauge measures. The protocol sequences. The Decision Interface acts.

Remark 8 (Exhaustiveness). Halt, Ask, and Act are exhaustive and mutually exclusive: every boundary evaluation produces exactly one action.

Every boundary satisfies exactly one of: (a) some decidable condition unsatisfied, (b) all surfaced with reducible uncertainty, (c) all conditions met and uncertainty below threshold. These are exhaustive and mutually exclusive by construction.

The separation is the design principle. Nothing in the gauge requires judgment. Nothing in the protocol requires domain knowledge. The moment a system gates on whether two URIs are "similar enough," it has contaminated the measurement with a decision. Measurement is reproducible—any agent running the gauge on the same codebooks gets the same result. Sequencing is deterministic—the dependency ordering is fixed. Judgment is auditable—the Decision Interface records *why* a particular action was chosen, not just that one was taken.

The current state of the world. Virtually every boundary between independently designed systems today has never been measured by this instrument. Shannon's metrics report perfect transmission. Every downstream framework reports convergence. The appropriate action at most boundaries is Halt—but instead, every system silently Acts, collapsing superposition by default without knowing it.

6.5 The Empty Context Principle.

The canonical claim form requires no prior agreement between systems—only that each system can articulate what *it* needs to make its own Meaning and Structure claims interpretable. This is local knowledge: every system already holds it about itself. The protocol does not require a system to know the other side's requirements before interaction begins. It surfaces both sides' requirements *through* the interaction.

A novel boundary—two systems that have never interacted—will have an empty Context layer. This is not a failure condition. It is the most important thing the protocol can tell you: no comparison on Meaning, Structure, or Data is valid until the Context layer contains sufficient resolutions to ground them.

The critical distinction is between two states that look identical from the outside but are categorically different. A system that ran the protocol and found no misalignment has *measured* coherence. A system that never ran the protocol has *unmeasured* uncertainty—and unmeasured uncertainty is not zero uncertainty. It is unbounded uncertainty: the system cannot distinguish alignment from misalignment, and it does not know what it is not measuring. This is the condition the protocol was built to surface. Existing interoperability tools require codebooks to already be exposed before they can operate. At a boundary where the protocol has never run, those tools have nothing to work with—and they do not report that condition. The coherence protocol makes the unmeasured state visible. You cannot know what you do not know you are missing—until you have an instrument that tells you the measurement was never taken.

7 Comparison to Existing Approaches

The coherence protocol is not the first system to address semantic conflict across boundaries. Context mediation Goh et al. [1999], ontology alignment Euzenat and Shvaiko [2007], schema matching, and the W3C semantic web stack all provide tools for reconciling representations across systems. The question is not whether prior work exists. It is where prior work stops.

7.1 Where Prior Work Operates

Every existing approach requires that both codebooks have already been surfaced before reconciliation can begin.

Context mediation Goh et al. [1999] defines explicit context axioms for each source—currency, units, tax inclusion—and uses abductive reasoning to reconcile conflicts automatically. But the axioms must be specified in advance, within a shared domain ontology. If a new source arrives whose context has not been modeled, the mediator has nothing to reason over.

Ontology alignment Euzenat and Shvaiko [2007] maps concepts between two exposed ontologies. Both ontologies must exist. Both must be accessible. The alignment algorithm operates on representations that have already been externalized. If a system has no ontology—if its assumptions are implicit in code, configuration, or convention—there is nothing to align.

Schema matching reconciles exposed schemas. It assumes both schemas are available and that the matching engine can access them. It operates on structure. It does not assess whether the meaning behind structurally similar fields is the same.

The W3C semantic web stack—RDF, OWL, SKOS, SHACL—provides the vocabulary for expressing codebooks. It does not provide the protocol for detecting whether two codebooks are aligned at a boundary. A system can have a comprehensive OWL ontology and still be in semantic superposition if the system on the other side uses different URIs for the same concepts—or the same URIs with different operational interpretations.

Each of these systems solves a real problem. None of them addresses the condition that precedes their operation: *ignorance of incoherence*—the state in which codebooks have not been surfaced, the Context layer is empty, the boundary has not been measured, and the system

does not know there is anything to reconcile. The coherence protocol operates at this layer: it provides the structured resolution process that must complete before any of these systems can do their work.

7.2 The Architectural Difference

Every prior system requires three separate components: a representation layer (where codebooks are stored), a measurement instrument (which reads and compares them), and a translation layer (which converts between the representation and the instrument's comparison frame). The ontology is not the alignment algorithm. The schema is not the matching engine. The context axioms are not the mediator. In each case, the instrument is separate from the substrate, and a translation step is required before comparison can begin.

The coherence protocol eliminates this separation. When System A projects its codebook onto the canonical claim form, and System B projects its codebook onto the same five-column surface, the gap between them is directly computable from the substrate itself. No separate instrument is required. No translation step intervenes. The canonical claim form is simultaneously the storage layer, the projection surface, and the comparison basis.

This is Liquid Coherence Compression: the elimination of the architectural redundancy between the substrate on which projections land and the instrument by which they are compared. In a rigid system—a fixed schema, a pre-negotiated ontology—these must be separate, because the schema constrains what can be projected, and a translation layer is required before comparison is possible. In a liquid system, no such constraint exists. The substrate conforms to whatever is projected onto it. The projection carries all the information needed for comparison. There is nothing left for a separate instrument to do.

7.3 Industry Need

The gap is not theoretical. It is operational.

Every API call between independently designed systems is a boundary crossing. Every database join across sources is a boundary crossing. Every LLM retrieval, every multi-agent message, every data pipeline hop is a boundary crossing. At each one, the coherence protocol has never been run. The codebook that gives the data its meaning stays on the sender's side. The receiver proceeds using its own.

Current industry practice addresses this with manual processes: data dictionaries, onboarding documentation, Slack conversations between teams, and tribal knowledge that lives in the heads of senior engineers. Each of these is an informal, unstructured form of Context—resolutions of incompleteness that were never recorded in a way that the next system, the next person, or the next agent can access. When those engineers leave, the Context leaves with them. When systems change, the documentation lags. When AI agents are given autonomy to cross boundaries their designers did not anticipate, no manual process intervenes.

The coherence protocol provides that instrument. It does not replace context mediation, ontology alignment, or schema matching. It operates at the layer below: accumulating the Context that determines whether codebooks have been surfaced at all, measuring the divergence when they have, and prescribing an action when they have not. Prior systems reconcile known conflicts. The coherence protocol builds the resolution layer that determines whether reconciliation is even possible—and whether, at this boundary, the system should Halt, Ask, or Act.

8 From Measurement to Computation on the Same Substrate

Section 6 showed that the canonical claim form was designed for measurement. But the structure has a deeper property that connects measurement, representation, and learning on a single

substrate. The gauge's outputs are not merely *compatible* with computation—they *are* computation, in a form that requires no transformation between human-readable assessment, formal verification, and machine learning.

We develop this in five stages: the Boolean diff, the tensor structure, the holonic property, the learning loop, and the connection to active inference.

8.1 The Boolean Diff

The gauge compares two systems' projections on the canonical claim surface by string equality. Every comparison produces a binary value: match (0) or mismatch (1). This is not a design choice that could have gone another way—it is a structural consequence of the claim form.

Consider two systems projecting onto the claim surface for a single field:

id	source	timestamp	key	value
`urn:trade:001`	`urn:sys:sgx`	`2026-03-10T09:00+08`	`urn:meaning:def`	execution time
`urn:trade:001`	`urn:sys:ldn`	`2026-03-10T01:00+00`	`urn:meaning:def`	receipt time

Table 5: Two systems' claims on the same field. The key matches. The values differ. Gauge output: 1 (mismatch). No interpretation required.

The gauge reads these two rows. The key matches (`urn:meaning:def`). The values differ. Output: 1. No interpretation was required. No similarity metric was applied. The comparison is string equality—the simplest possible operation—and the output is a single bit.

At each boundary, the gauge produces a structured output. The resolution layer yields a single completion bit:

$$\mu_C \in \{0, 1\} \tag{6}$$

where $\mu_C = 1$ iff all resolutions declared as required by both systems' Meaning and Structure claims have been recorded in Context. The three comparison facets yield:

$$\vec{\mu}_{\text{comp}} = (\mu_M, \mu_S, \mu_D) \in \{0, 1\}^3 \tag{7}$$

where $\mu_M = 1$ iff Meaning definitions mismatch; $\mu_S = 1$ iff Structure definitions mismatch; and $\mu_D = 1$ iff data values differ under aligned codebooks. The comparison vector is defined only in dependency order: μ_M, μ_S are valid comparisons only when $\mu_C = 1$; μ_D is interpretable only when $\mu_C = 1$, $\mu_M = 0$, $\mu_S = 0$. The full boundary state is $(\mu_C, \vec{\mu}_{\text{comp}})$, where μ_C gates the validity of the comparison vector. Across K boundaries, the comparison-facet state space is 2^{3K}; the effective state space is larger when multiple Context resolution steps are required per boundary.

This vector is not an encoding of the measurement. It *is* the measurement. The same object that a human reads as "Context: unsurfaced, Meaning: mismatch, Structure: match" is also a literal binary vector that requires no transformation for any downstream computation.

We call this the **Boolean diff**: the gauge's native output format is a diff between two codebook projections, expressed in pure Boolean form, produced without any encoding, discretization, or feature engineering step.

8.2 The Tensor Structure

For K boundaries, each with three comparison facets and a Context completion bit, the boundary configuration is:

$$M \in \{0, 1\}^{K \times 4} \tag{8}$$

This is a binary matrix. For a system monitoring N fields across K boundaries over T time steps, the full state is:

$$\mathcal{M} \in \{0, 1\}^{T \times K \times N \times 4} \tag{9}$$

This tensor is the native language of machine learning. No transformation is needed. The gauge outputs are already tensors. The claim surface is already a feature space.

This connects directly to Tensor Logic Domingos [2025] and its JSON-TL specification Itelman [2026b].

```
Y[i] = step(W[i, j] X[j])
```

The input tensor $X[j]$ can be a flattened boundary state vector—the gauge outputs for all facets at a boundary. The weight tensor $W[i, j]$ learns which facet patterns predict which outcomes. The output $Y[i]$ classifies the boundary state into resolution categories. The entire computation is expressible in JSON-TL's 17 token types and 10 AST node types.

Layer		**Role**
Canonical Claim Surface	$\rightarrow$	produces the data
Boolean Diff (Gauge)	$\rightarrow$	produces $\{0, 1\}$ vectors
Tensor Structure	$\rightarrow$	organizes vectors into $\{0, 1\}^{T \times K \times N \times 4}$
Tensor Logic	$\rightarrow$	grammar for computing on tensors
JSON-TL	$\rightarrow$	machine-readable specification for the grammar

Figure 8: The computational pipeline from measurement to inference. Each layer produces the input for the next. No transformation step intervenes between any two layers.

The claim surface produces the data. Tensor Logic provides the grammar for computing on it. JSON-TL provides the machine-readable specification. These are not three separate systems that must be integrated—they share the same structural foundation: tensors over binary values, with identity (URIs) at every node.

8.3 The Holonic Property

The architecture exhibits a property that Koestler [1967] identified as fundamental to complex adaptive systems: each component is simultaneously a whole in its own right and a part of a larger structure. Koestler called such entities *holons* and the nested structure they form a *holarchy*.

A single gauge comparison is a holon. It is a complete measurement—a definite binary output, human-readable, auditable, sufficient to trigger a protocol action. It is also a single element in a boundary state vector that participates in tensor operations across the full system.

A boundary state vector $\vec{\mu} \in \{0, 1\}^4$ is a holon. It is a complete assessment of one boundary—enough to determine Halt, Ask, or Act. It is also one row in the configuration tensor M that spans an entire pipeline.

A trained classifier at one boundary is a holon. It operates autonomously—predicting misalignment patterns, recommending Ask sequences, learning from local resolution history. It is also a node in the context graph, contributing its predictions to global pattern detection and accessible to other boundaries facing similar configurations.

The context graph itself is a holon. It is a complete record of boundary states across the system—sufficient for organizational-level coherence monitoring. It is also a component of a larger inter-organizational graph when systems cross institutional boundaries.

No level cancels the others. The gauge output does not stop being human-readable because it is also a tensor feature. The local classifier does not stop being autonomous because it also feeds a global model. The boundary measurement does not stop being a complete assessment because it is also a training example. This is Koestler's point: the real structure of complex systems is neither atomistic (reduce everything to parts) nor holistic (only the whole matters)—it is holarchic, with genuine autonomy at every level and genuine integration across levels.

The canonical claim form enables this because it provides identity (URIs), provenance (source, timestamp), and structural uniformity (five fixed columns) at every level simultaneously. A gauge output is a claim. A classifier's weights are claims. A learned pattern stored back into the graph is a claim. The substrate is the same at every level of the holarchy.

8.4 The Learning Loop

Because the gauge outputs are clean binary tensors with URI-identified nodes, the system supports a learning loop that feeds back into the context graph at two scales.

Local learning. Two systems that interact frequently accumulate a history of gauge outputs at their shared boundary. A classifier trained on this history learns the specific patterns of that boundary: which facets tend to mismatch, which Ask sequences resolve mismatches most efficiently, which time periods introduce drift. This trained model is stored as claims in the context graph—its weights, its provenance, its training history, all expressed in the canonical form. It becomes part of the boundary's Context: future protocol runs at this boundary can consult the learned model before issuing Asks.

Global learning. Across all boundaries in the graph, the configuration tensor $\mathcal{M}$ accumulates structural patterns. Tucker decomposition of this tensor reveals which facet combinations co-occur across boundaries, which system types are prone to specific misalignment patterns, and which resolution strategies generalize. A globally trained model—also stored as claims in the canonical form—provides predictions for new boundaries that have no local history. When System A encounters System B for the first time, the global model can predict the most likely misalignment pattern based on the system types, domains, and configurations it has seen elsewhere.

Self-referential hosting. The models live on the same substrate as the data they were trained on. A perceptron's weights are claims:

id	source	timestamp	key	value
`urn:model:bnd-sgx-ldn`	`urn:sys:coh`	`2026-03-15`	`urn:model:w:0,0`	0.73
`urn:model:bnd-sgx-ldn`	`urn:sys:coh`	`2026-03-15`	`urn:model:w:0,1`	−0.41
`urn:model:bnd-sgx-ldn`	`urn:sys:coh`	`2026-03-15`	`urn:model:meta:acc`	0.94
`urn:model:bnd-sgx-ldn`	`urn:sys:coh`	`2026-03-15`	`urn:model:meta:src`	`urn:bnd:sgx-ldn`

Table 6: A trained perceptron's weights stored as canonical claims. The model has identity (a URI), provenance (source, timestamp), and a place in the context graph. It can be versioned, audited, and compared on the same substrate as the measurements it optimizes.

This closes the loop: the context graph measures boundaries, the measurements become training data, the trained models feed back into the graph as boundary-level intelligence, and the graph's predictive capacity improves with each measurement cycle. The graph does not just record coherence—it learns where coherence breaks and anticipates it.

In Tensor Logic notation, the local classifier at a boundary is:

$$Y[i] = \text{step}(W[i,j]\ X[j]) \tag{10}$$

where $X[j]$ is the flattened gauge output vector $\vec{\mu}$ for that boundary, $W[i,j]$ are the learned weights (stored as canonical claims), and $Y[i]$ is the predicted resolution category. The JSON-TL specification Itelman [2026b] provides the machine-readable token and AST vocabulary for parsing this equation, enabling browser-based tooling, API integration, and cross-platform execution of boundary classifiers.

8.5 Connection to Active Inference

This connects to active inference Friston [2010] in a precise way. An active inference agent selects actions that maximally reduce expected free energy. The coherence protocol's Ask action selects the facet whose resolution maximally reduces boundary uncertainty. If the learning loop provides a model that predicts, from the current boundary state vector, which Ask will yield the greatest entropy reduction, then the protocol becomes an active inference policy over the context graph.

The formalization is direct:

- **Hidden state**: the boundary configuration tensor $M \in \{0,1\}^{K\times 4}$
- **Action space**: the set of possible Asks (which facet, at which boundary)
- **Observation**: the gauge output after each Ask
- **Policy**: select the Ask that maximally reduces expected boundary entropy, as predicted by the learned model

The protocol measures coherence. The learning loop optimizes measurement. The active inference framing provides the decision-theoretic grounding for *which* measurement to take next. All three—measurement, learning, and policy—operate on the same canonical claim substrate.

The result is a recursive, self-referential system: the context graph measures boundaries, the inference layer optimizes measurement, and both are expressed as claims in the canonical form. The graph community, the semantics community, and the active inference community converge on the same substrate. Formalizing this convergence is the subject of ongoing work.

9 Why a Standard

The coherence protocol measures a property that no existing standard measures. This section states the specific claims that motivate standardization, each grounded in results established in prior sections.

The problem is formally proven, not asserted. Codebook alignment is not identifiable from channel observables (Observation 1). This is a mathematical property of Shannon's model, not an engineering opinion. Any standard built on transmission metrics alone inherits this blind spot. A standard for boundary coherence fills a gap that is provably unfillable by existing instruments.

Every existing standard assumes the precondition this protocol measures. Schema matching, ontology alignment, context mediation, SHACL validation, data contracts, and API specifications all require that codebooks have been surfaced before reconciliation can begin (Section 7). The coherence protocol operates at the layer below: detecting whether that precondition holds. A W3C standard for boundary coherence does not compete with RDF, OWL, or SHACL—it provides the verification layer they currently lack.

AI agents cross boundaries without human intervention . A single agent query may touch ten tools, ten APIs, ten systems—each with its own codebook. Today, when a human encounters an unfamiliar field, they ask what it means—they build Context. Agents infer and proceed with confidence. Their Context layer is empty. The more capable the agent, the more convincingly it proceeds with a wrong interpretation. A standard protocol for surfacing and comparing codebooks at boundaries gives agents the equivalent of picking up the phone—a structured way to build the Context that coherence requires.

The measurement is domain-agnostic. The canonical claim form (Definition 7) requires no prior agreement on domain terms. The five columns are fixed; the namespace is open. Any system in any domain can project its codebook onto the same surface. This is the property that makes standardization feasible: the standard specifies the envelope, not the message.

The cost is bounded. Facet comparison is $O(1)$ per field per boundary. The gauge requires only string equality—no similarity metrics, no inference, no domain knowledge. Any agent running the gauge on the same projections gets the same result. This reproducibility is what makes the measurement auditable and the standard enforceable.

The gap is operational now. Every API call between independently designed systems, every database join across sources, every multi-agent workflow is a boundary crossing where the coherence protocol has never been run. Organizations are not waiting for a theoretical motivation. They are waiting for an instrument.

10 Holonic Coherence

The protocol comprises four layers: Substrate, Ontology, Coherence, and Decision (Figure 1). Each layer is self-contained—it performs a complete function under its own logic. Each layer is also a component of the layers above it. This is the holonic property: every layer is simultaneously a whole and a part, and no layer's autonomy is sacrificed by its integration into the larger system.

10.1 The Four Layers

Substrate: the canonical claim form. Five columns. URN-namespaced keys. Every assertion from any system reduces to a single row (Definition 7). The substrate is a whole: it is a complete, self-describing record—any claim can be read, compared, and audited without reference to the layers above. It is also a part: it provides the projection surface on which the coherence layer operates and the feature space on which the inference layer learns.

Ontology: the domain knowledge layer. Existing semantic web standards—RDF, OWL, SHACL—express the vocabulary and constraints of a domain. This layer is a whole: a comprehensive OWL ontology is a complete, self-consistent knowledge system. It is also a part: when a system projects its codebook onto the claim surface, the ontology determines what claims are generated. The coherence protocol does not replace this layer. It consumes its output.

Coherence: the boundary measurement and resolution layer. The three comparison facets, the Context resolution layer, the dependency ordering, and the Halt/Ask/Act decision interface. This layer is a whole: a single boundary assessment is a complete measurement with a definite action—Context accumulated, comparisons evaluated, protocol state recorded. It is also a part: each assessment contributes a binary vector to the configuration tensor $M \in \{0,1\}^{T \times K \times N \times 4}$ that spans the full system.

Decision: the systems that act on information. Bayesian inference, active inference, dynamic programming, human judgment—whatever framework selects actions given interpreted data. This layer is a whole: the decision framework operates under its own logic and optimality criteria. It is also a part: the validity of its output depends on the coherence layer having verified that the information it operates on means what it thinks it means.

10.2 How the Layers Compose

The dependency flows upward. The substrate provides the projection surface. The ontology populates it. The coherence layer measures it. The decision layer acts on the result. Each layer can operate independently—a claim can be stored without being measured, a measurement can be taken without a decision being made—but the reliability of each higher layer depends on the layers below it having done their work.

This is the same dependency ordering that governs the facets within a single boundary (Context → Meaning → Structure → Data), now applied to the architecture itself: Substrate → Ontology → Coherence → Decision. A decision made without coherence is like a data comparison made without accumulated Context—it may be correct, but its correctness is unverifiable, because the resolutions that would make verification possible were never recorded.

10.3 The Learning Loop Across Layers

The holonic property enables a feedback loop that crosses layers without violating their autonomy.

The coherence layer produces gauge outputs: binary vectors. These vectors are substrate-native—they are already in the canonical claim form. A classifier trained on accumulated gauge outputs learns boundary patterns: which system pairs mismatch on Meaning, which facet combinations predict failures, which boundaries need more Asks. The trained model is stored back into the substrate as claims—its weights, its provenance, its training history, all expressed in the same five-column form.

The model is now available to the coherence layer: future protocol runs at a boundary can consult the learned model before issuing Asks. It is also available to the decision layer: an agent can query the context graph to assess, before crossing a boundary, whether coherence is likely to hold or likely to require intervention.

No layer is modified to support this loop. The substrate stores claims—it does not know or care whether a claim is a raw data value, an ontology assertion, a gauge output, or a model weight. The coherence layer compares claims by string equality—it does not know or care whether the claims it compares were generated by a system projection or a trained classifier. The decision layer acts on coherent information—it does not know or care how coherence was achieved.

Each layer does its own job. The holonic composition is what makes the system learn.

10.4 What This Does Not Claim

This architecture does not claim that the holonic property is unique to this system. Holonic structure appears in biological systems, organizational design, and software architecture.

What we claim is narrower and specific: the canonical claim form's five-column design produces a substrate where measurement, representation, and training data are the same object—with no transformation step between any of these uses—and where models trained on the system's own outputs can be stored back into the system in the same format as the data they were trained on. This self-referential property is a consequence of the substrate design, not an additional feature bolted onto it.

Whether this property is genuinely novel or has precedent in existing semantic interoperability frameworks is an open question we pose to the community.

11 Limitations

This paper presents a framework and a protocol, not an empirical evaluation. The four-facet decomposition, canonical claim form, and Halt/Ask/Act regime are defined formally and demon-

strated on worked examples, but they have not been benchmarked against existing systems on a shared dataset.

The upper bound on boundary uncertainty ($3K$ bits from comparison facets for K boundaries, plus boundary-dependent Context resolution costs) is a combinatorial fact under a maximum-ignorance prior. Real-world boundaries will have prior information that lowers the effective uncertainty.

The $O(1)$ cost claim applies to facet comparison once codebooks have been surfaced. The cost of surfacing—projecting implicit assumptions onto the canonical form—is system-dependent and may be substantial.

The protocol requires that participating systems agree to externalize their codebooks. The adoption problem is separate from the measurement problem.

The learning loop and active inference formalization described in Section 8 are theoretical projections. Implementation, formal proofs of convergence, and empirical validation are future work.

12 Future Work

Empirical evaluation. Benchmark the protocol against schema matching, ontology alignment, and context mediation systems on shared datasets. Measure false-positive and false-negative rates for boundary misalignment detection. Quantify surfacing costs across system types and domains.

Tucker decomposition at scale. Decompose $\{0,1\}^{T\times K\times N\times 4}$ tensors from production boundary networks. Identify organizational-level coherence signatures—structural patterns in misalignment that reveal systemic issues rather than boundary-specific ones.

JSON-TL integration. Implement the learning loop using JSON-TL Itelman [2026b] as the computational grammar. Demonstrate that boundary classifiers can be expressed as Tensor Logic equations, parsed via JSON-TL, and executed on gauge outputs with no intermediate representation.

Self-referential model hosting. Implement perceptron-based classifiers whose weights are stored as canonical claims, demonstrating end-to-end self-referential inference: measurement $\rightarrow$ training $\rightarrow$ model-as-claims $\rightarrow$ improved measurement.

Active inference formalization. Formalize the connection between the Ask action and expected free energy minimization. Prove that the optimal Ask sequence under the learned model converges to minimum boundary entropy.

W3C specification. Develop the canonical claim form as a W3C specification for boundary coherence. Investigate integration with RDF-Star and SHACL 1.2—particularly whether the CBox (contextual metadata layer) provides the right infrastructure for housing boundary coherence measurements and learned models within the semantic web stack.

Holarchic scaling. Formalize the holonic property: prove that the autonomy of each level (gauge output, boundary vector, local model, global model, context graph) is preserved under composition, and that no level's operation depends on the existence of higher levels.

13 Conclusion

A receiver gets a value. The channel is perfect. The bits are correct. The receiver is in ignorance of incoherence—it proceeds with zero uncertainty about an interpretation that may be wrong.

Every framework that minimizes uncertainty presupposes that the codebook is shared. We proved that no quantity computable from Shannon's model detects codebook misalignment (Observation 1). We showed the same gap extends to active inference, Bayesian decision theory, and dynamic programming.

We provided the instrument. The codebook decomposes into three comparison facets—Meaning, Structure, and Data—and a resolution layer, Context, that accumulates the information required to make those comparisons well-defined. Context is not a fourth dimension alongside the others. It is the protocol's working memory: the structured record of every incompleteness discovered, every question asked, and every resolution achieved. The canonical claim form—five columns, URN-namespaced facets, liquid assembly—provides the irreducible substrate onto which any system can project its codebook. The coherence protocol classifies every boundary into Halt, Ask, or Act, and collapses semantic superposition at $O(1)$ per facet comparison per boundary. Every Ask produces Context. Every Context entry resolves an incompleteness. The protocol terminates when no further incompleteness is visible to either party. That is the coherence condition.

The substrate reduces to binary. Binary admits tensor decomposition. Tensor decomposition admits inference. The same five-column form that measures coherence can host the models that optimize measurement. The graph, the semantics, and the inference converge on one substrate.

The instrument now exists. The Context layer it builds was always needed. Every boundary that lacks it is in ignorance of incoherence. The protocol makes that ignorance visible—and resolvable.

References

Albert-László Barabási. Personal communication, March 2026.

Richard E. Bellman. *Dynamic Programming.* Princeton University Press, Princeton, NJ, 1957.

Pedro Domingos. *The Master Algorithm.* Basic Books, New York, 2015.

Jérôme Euzenat and Pavel Shvaiko. *Ontology Matching.* Springer, Heidelberg, 2007.

Karl Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

Cheng Hian Goh, Stéphane Bressan, Stuart E. Madnick, and Michael D. Siegel. Context interchange: New features and formalisms for the intelligent integration of information. *ACM Transactions on Information Systems*, 17(3):270–293, 1999.

Ron Itelman and Juan Cruz Viotti. *Unifying Business, Data, and Code: Designing Data Products with JSON Schema.* O'Reilly Media, 2024.

Leonard J. Savage. *The Foundations of Statistics.* John Wiley & Sons, New York, 1954.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

Warren Weaver. Recent contributions to the mathematical theory of communication. In Claude E. Shannon and Warren Weaver, editors, *The Mathematical Theory of Communication*, pages 1–28. University of Illinois Press, Urbana, 1949.

Pedro Domingos. Tensor Logic: The Language of AI. *arXiv:2510.12269v3 [cs.AI]*, 2025.

Ron Itelman. JSON-TL: Token and AST Specification for Tensor Logic. Version 1.0.0-draft, 2026.

Arthur Koestler. *The Ghost in the Machine.* Hutchinson, London, 1967.